



Módulo 2 · Repaso y práctica de lo visto en clase

Creación de Funciones y Algoritmos con IA

Ejercicio de Clase

Nombre del alumno:

Angel David Echenique Villanueva / Z0271067V

Fecha:

09/03/2026

Instrucciones

Responde con lo que vimos en clase. Puedes consultar tus apuntes. No hay que escribir código, solo demostrar que entendiste!

BLOQUE
E 1

Funciones simples vs. lógica compleja

" No pidas 'un algoritmo de ordenamiento'; pide una función que procese un array de objetos Task, valide fechas y maneje nulos. "

Ejercicio 1.1 Clasifica cada tarea

En clase vimos que la IA es muy eficaz en ciertos tipos de funciones. Escribe S (Simple) o C (Compleja) según el tipo al que pertenece cada tarea:

S / C	Tarea
S	Validar el formato de un email con regex
C	Procesar un array de transacciones con múltiples monedas y tasas de conversión
S	Convertir temperaturas de Celsius a Fahrenheit
C	Gestionar permisos y roles de usuarios en un sistema
S	Capitalizar la primera letra de cada palabra en un string
C	Crear un parser de expresiones matemáticas con paréntesis y variables
S	Calcular el promedio de una lista de números
C	Implementar lógica de negocio para analizar logs del servidor

Ejercicio 1.2 El prompt completo para funciones simples

En clase vimos que cuando pedimos una función simple a la IA, hay 4 elementos que siempre deberíamos incluir en el prompt. Escríbelos:

1.	Nombre o proposito de la funcion.
2.	Inputs son los parametros de entrada.
3.	Output esperado
4.	Lenguaje y contexto de programacion.

Ejemplo de como usarlo en un prompt cortito:

Crea una función en Python que reciba una lista de números y devuelva el promedio.

 BLOQUE 2	Algoritmos avanzados	
---	-----------------------------	--

En clase vimos que la IA puede generar algoritmos avanzados de tres grandes familias. Cada una con sus algoritmos y estructuras propias.

Ejercicio 2.1 Completa las tres familias de algoritmos

Escribe los algoritmos y estructuras que pertenecen a cada familia, según lo que vimos en clase:

Familia	Algoritmos / Estructuras que la componen
Búsqueda	BFS, DFS, A*, búsqueda binaria.
Ordenamiento	QuickSort, MergeSort, HeapSort, BubbleSort.
Estructuras de datos	Trie, Árboles, Hash tables, Grafos, Pilas y Colas.

Ejercicio 2.2 Une el algoritmo con su descripción

Une cada algoritmo con lo que hace:

BFS (Búsqueda en Anchura)	→	Explora el camino de menor coste usando heurística para guiar la búsqueda
DFS (Búsqueda en Profundidad)	→	Divide el array en mitades y las ordena recursivamente antes de unir las
A*	→	Explora todos los nodos a la misma distancia antes de ir más lejos
QuickSort	→	Explora un camino completo hasta el final antes de retroceder
MergeSort	→	Elige un pivote y coloca los menores a la izquierda y los mayores a la derecha
Trie	→	Estructura de árbol optimizada para búsquedas de cadenas de texto (autocompletado)

Ejercicio 2.3 Verdadero o Falso

Marca si cada afirmación sobre algoritmos avanzados es Verdadera (V) o Falsa (F):

V / F	Afirmación	Nº
V	La IA puede implementar algoritmos como A* o QuickSort si se lo pedimos correctamente.	1
F	No hace falta especificar el lenguaje cuando pedimos un algoritmo avanzado a la IA.	2
V	Validar la complejidad temporal del código generado por la IA es responsabilidad del programador.	3
V	Un Trie es una estructura de datos útil para autocompletado y búsqueda de texto.	4
F	La IA siempre genera la implementación más eficiente de un algoritmo, sin necesidad de indicarlo.	5
V	Podemos pedirle a la IA que explique paso a paso cómo funciona el algoritmo que genera.	6



BLOQUE
E 3

Estrategia de 3 niveles para crear funciones

En clase vimos que pedir código a la IA de forma profesional sigue una estrategia de 3 niveles según la complejidad de lo que necesitamos.

Ejercicio 3.1 Describe cada nivel

Completa la tabla con lo que caracteriza a cada nivel y un ejemplo del módulo:

Nivel

1

Funciones simples

¿Qué información hay que incluir en el prompt de este nivel?

Decirle que hará la función, especificar el lenguaje, especificarle entrada y salida.

Nivel

2

Lógica compleja

¿Qué información hay que incluir en el prompt de este nivel?

Tenemos que describir bien los problemas, estructura de datos, las reglas de negocios, casos límites y lenguaje.

Nivel

3

Sistemas completos

¿Qué información hay que incluir en el prompt de este nivel?

Aquí es donde ya pasamos a pedir todo paso a paso, comenzamos con arquitectura de sistema, componentes, comunicación entre módulos, escalabilidad, lenguaje y tecnología específicas.

Ejercicio 3.2 ¿A qué nivel pertenece este prompt?

Lee cada prompt y escribe el nivel (1, 2 o 3) que le corresponde según la estrategia vista en clase:

Nivel (1/2/3)	Prompt
<u>1</u>	"Función que capitalice la primera letra de cada palabra."
<u>3</u>	"Diseña un sistema de caché distribuido con estrategia LRU, replicación y failover automático en Go."

2	"Parser de expresiones matemáticas con soporte para paréntesis, operadores y variables."
1	"Función que filtre una lista de números y devuelva solo los pares."
2	"Implementa un sistema de permisos por roles con herencia, validación y auditoría de accesos."
1	"Crea una función que convierta temperaturas de Celsius a Fahrenheit."



Buenas prácticas al generar algoritmos con IA

BLOQUE 4

En clase vimos 6 buenas prácticas que debemos aplicar siempre que pedimos a la IA que genere un algoritmo o función.

Ejercicio 4.1 Recuerda las 6 buenas prácticas

Escribe las 6 buenas prácticas del módulo y explica brevemente para qué sirve cada una:

Buena práctica	¿Para qué sirve?
1. especificar lenguaje	Asi evitamos ambigüedades
2. definimos inputs y outputs	Garantizamos que la funcion haga lo esperado
3. pedimos complejidad algoritmica	Aseguramos eficiencia
4. le pedimos que nos explique el algoritmo implementado	Nos permite entender el codigo y que hace.
5. Cases edge	Esto nos ayuda a evitar errores en situaciones limites
6. validar el proyecto con test.	Con eso garantizamos que todo funciona correctamente.

Ejercicio 4.2 ¿Buena práctica o error común?

Marca BP (Buena Práctica) o EC (Error Común) según lo visto en clase:

BP / EC	Afirmación
BP	Pedir al prompt que especifique la complejidad algorítmica deseada, ej. $O(n \log n)$.
EC	Aceptar el primer algoritmo que genera la IA sin compararlo con implementaciones estándar.
BP	Solicitar a la IA que muestre casos límite (edge cases) del algoritmo generado.
EC	Pedir todo el sistema en un solo prompt para ahorrar tiempo.
BP	Pedir a la IA que explique el algoritmo paso a paso antes de usarlo.
EC	No ejecutar tests sobre el código generado porque la IA raramente se equivoca.
BP	Especificar si no se pueden usar librerías externas o hay restricciones de memoria.
BP	Validar el código generado con tests unitarios antes de integrarlo en el proyecto.
 BLOQ UE 5	Pregunta de síntesis

Un compañero necesita una función que procese pedidos de una tienda online. Te enseña el prompt que va a usar:

"Hazme una función para pedidos."

1. ¿A qué nivel de la estrategia de 3 niveles pertenece esta función? ¿Por qué no encaja en el Nivel 1? = Es de nivel 2, porque tiene lógica compleja, ya que procesar pedidos de una tienda, implica múltiples reglas.
2. Reescribe el prompt correctamente aplicando la estrategia del nivel que corresponde. Incluye inputs, outputs y al menos un caso límite:

✓ **Mi prompt mejorado:**

Creame una funcion en python que procese pedidos de una tienda online donde me incluyas:

Inputs:

- lista de productos con precio y cantidad
- porcentaje de descuento
- impuesto aplicable

Output:

- total del pedido
- lista de productos procesados

Requisitos:

- validar que las cantidades sean mayores que 0
- calcular subtotal, descuento e impuestos

Caso límite:

- si la lista de productos está vacía, devolver total = 0.

3. ¿Qué buenas prácticas aplicarías DESPUÉS de que la IA te genere el código?

Menciona al menos 3:

1-) Primero que nada revisar la complejidad del algoritmo.

2-) Probar todo con test unitarios.

3-) Validaciones de casos limites.

Por ultimos revisar la logica paso a paso.